Task 5: Transaction-Based Payment Simulation

Aim: To simulate a secure payment transaction between a user and a merchant using database operations. To ensure data consistency by committing successful transactions and rolling back failed ones**.**

Algorithm:

1. Start
2. Get userId, merchantId, and amount from input
3. Connect to database
4. Start database transaction (BEGIN)
5. Fetch user balance from database
6. Check if user exists
7. If user does not exist → stop and show error
8. Check if user balance ≥ amount
9. If insufficient balance → stop and show error
10. Deduct amount from user account
11. Add amount to merchant account
12. Insert transaction record with status = "SUCCESS"
13. Commit the transaction (COMMIT)
14. Display "Payment Successful"

PROGRAM:

Index.html

```
<!DOCTYPE html>
<html>
<head>
   <title>Payment Simulation</title>
</head>
<body>
<h2>Transaction System</h2>
<input id="userId" placeholder="User ID">
<input id="merchantId" placeholder="Merchant ID">
<input id="amount" placeholder="Amount">
```

```
<button onclick="pay()">Pay</button>
<p id="result"></p>
<script>
async function pay() {
   const response = await fetch("http://localhost:3000/transfer", {
      method: "POST",
      headers: {
         "Content-Type": "application/json"  },
      body: JSON.stringify({
         userId: document.getElementById("userId").value,
         merchantId: document.getElementById("merchantId").value,
         amount: document.getElementById("amount").value
      })
   });
   const data = await response.json();
   document.getElementById("result").innerText = data.message;
}
</script>
</body>
</html>
```

Db.js

```
const mysql = require("mysql2");
const pool = mysql.createPool({
   host: "localhost",
   user: "root",
   password: "MySQL@1234",
   database: "payment_db"
}
);
module.exports = pool.promise();
```

Package.json

```
{
  "name": "payment-simulation",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "body-parser": "^2.2.2",
    "cors": "^2.8.6",
    "express": "^5.2.1",
    "mysql2": "^3.22.0"
  }
}
```

Server.js

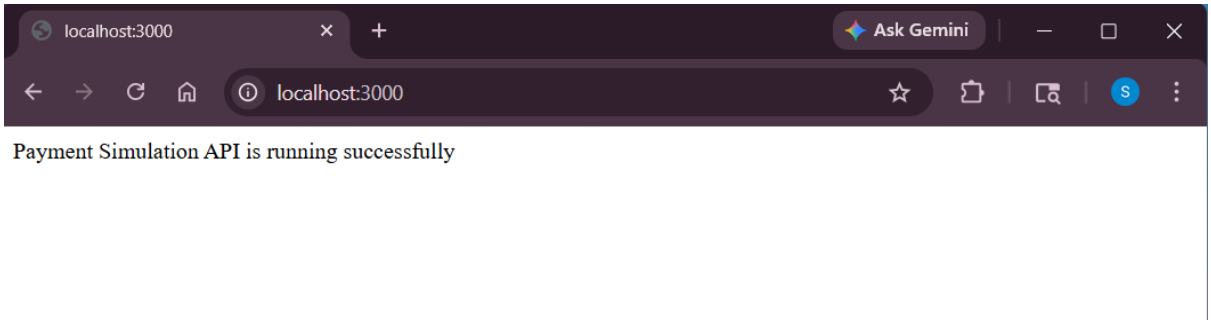
```
const express = require("express");
const db = require("./db");
const cors = require("cors");
const app = express();
app.use(express.json());
app.use(cors());
app.post("/transfer", async (req, res) => {
    const { userId, merchantId, amount } = req.body;
    const connection = await db.getConnection();
```

```
  try {
    await connection.beginTransaction();
    // 1. Check user balance
    const [rows] = await connection.query(
      "SELECT balance FROM users WHERE id = ?",
      [userId]
    );
    if (!rows.length || rows[0].balance < amount) {
      throw new Error("Insufficient balance");
    }
    // 2. Deduct from user
    await connection.query(
      "UPDATE users SET balance = balance - ? WHERE id = ?",
      [amount, userId]
    );
    // 3. Add to merchant
    await connection.query(
      "UPDATE merchants SET balance = balance + ? WHERE id = ?",
      [amount, merchantId]
    );
    // 4. Commit transaction
    await connection.commit();
    res.send({ message: "Payment Successful" });
  } catch (err) {
    await connection.rollback();
    res.status(400).send({ message: err.message });
  } finally {
    connection.release();
  }
});
```

```
app.get("/", (req, res) => {
  res.send("Payment Simulation API is running successfully");
});
// ✅ IMPORTANT: server should start ONLY ONCE here
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Output:

```
Default

Enter password: **********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 377
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statemen
t.

mysql> CREATE DATABASE payment_db;
Query OK, 1 row affected (0.13 sec)

mysql> USE payment_db;
Database changed
mysql> CREATE TABLE users(
    -> id INT PRIMARY KEY AUTO_INCREMENT,
    -> name VARCHAR(100),
    -> balance DECIMAL(10,2) NOT NULL
    -> );
Query OK, 0 rows affected (0.21 sec)

mysql> CREATE TABLE merchants (
    -> id INT PRIMARY KEY AUTO_INCREMENT,
    -> name VARCHAR(100),
    -> balance DECIMAL(10,2) NOT NULL
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO users(name, balance)
    -> VALUES ('User1' , 5000);
Query OK, 1 row affected (0.05 sec)

mysql> INSERT INTO merchants (name, balance)
    -> VALUES ('Merchant1', 1000);
Query OK, 1 row affected (0.01 sec)
```



Result:

Payment is processed by deducting money from the user and adding it to the merchant. If all steps succeed, the transaction is committed successfully. If any step fails, the transaction is rolled back to maintain consistency.